

<https://gemini.google.com/share/484d47bb91e8>



Temel Sıralama Algoritmaları

Hafta 2

<https://gemini.google.com/share/484d47bb91e8>

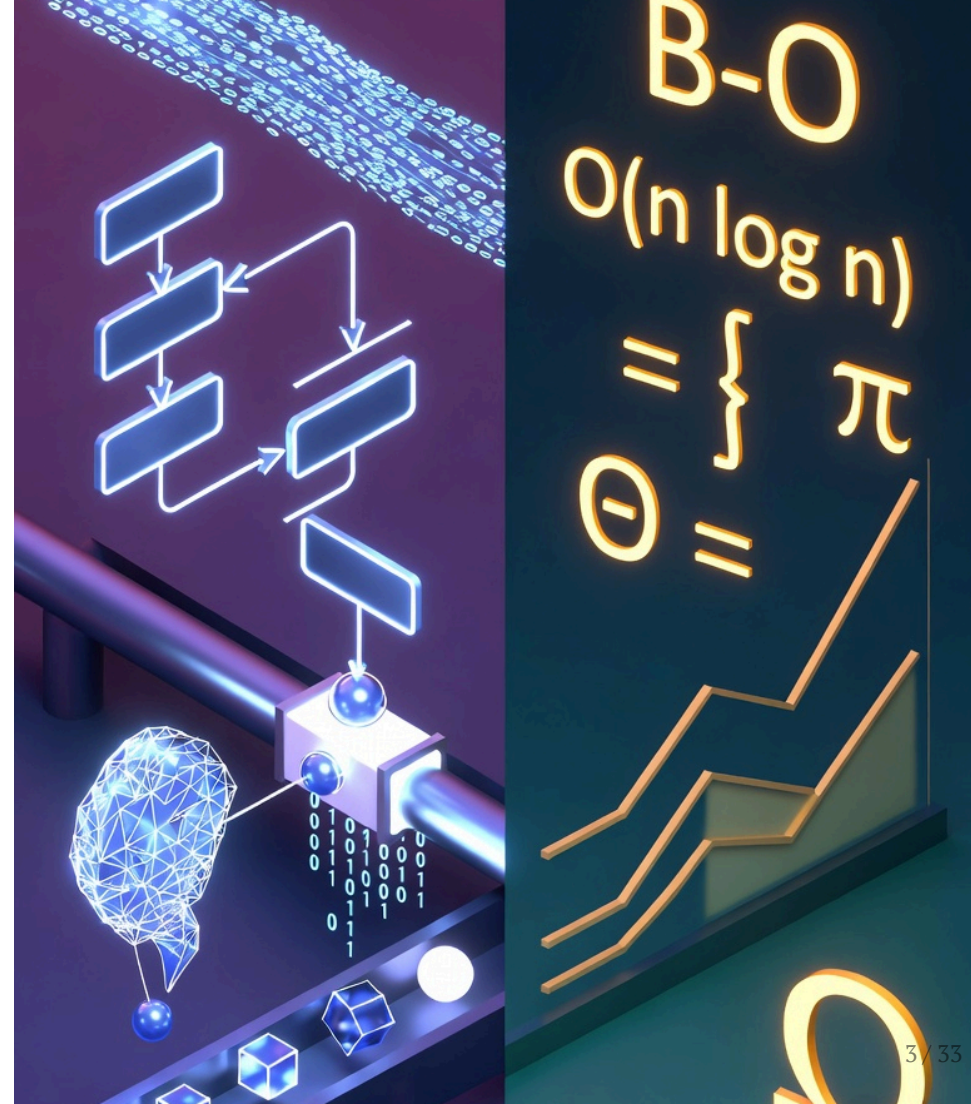
Sıralama Algoritmalarına Giriş

- Sıralama problemi, dağınık veriyi belirli bir anahtara göre düzenleme işlemidir.
- Düzenli veri, arama operasyonlarını lineer zamandan logaritmik zamana ($O(\log n)$) indirger.
- Bu hafta incelenecek algoritmalar temel karesel zaman, yani $O(n^2)$ sınıfındadır.
- Amacımız karmaşık yapılar öncesi temel 'algoritmik sezgi' ve analitik beceri kazanmaktır.

Sıralama Kriterleri: Stability ve In-Place

- Kararlılık (Stability): Eşit değerdeki elemanların başlangıçtaki bağıl sıralarını korumasıdır.
- Çoklu anahtarla (isim, sonra tarih) sıralama yaparken kararlılık hayati önem taşır.
- Yerde Sıralama (In-Place): Algoritmanın giriş dizisi dışında ekstra bellek kullanmamasıdır.
- In-Place algoritmalar $O(1)$ alan karmaşıklığı ile bellek kullanımını minimize ederler.

<https://gemini.google.com/share/484d47bb91e8>



Selection Sort: Temel Felsefe

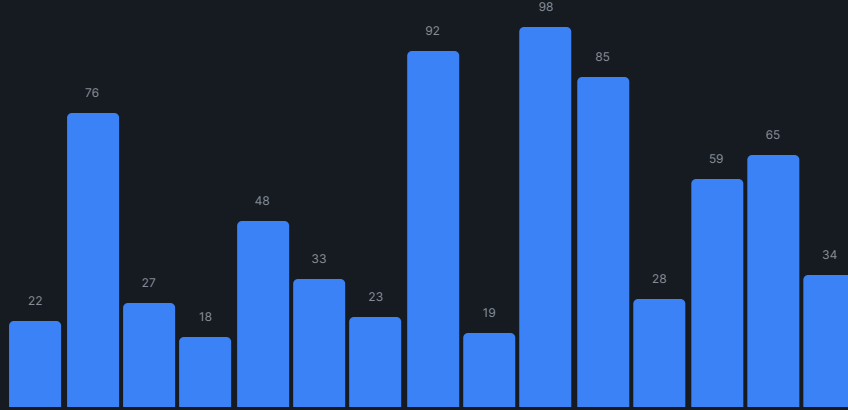
- Selection Sort, her turda dizideki en küçük elemanı arayıp bularak çalışır.
- Bulunan minimum eleman, sırasız bölümün en başındaki elemanla yer değiştirilir (swap).
- Dizi mantıksal olarak ikiye ayrılır: Sol taraf sıralı, sağ taraf sırasız bölge.
- Basit yapısı ve insan sezgisine yakınlığı ile algoritmik düşünmenin temelidir.

<https://gemini.google.com/share/484d47bb91e8>

Selection Sort

İteratif olarak dizideki en küçük elemanı bulur ve mevcut sıradaki elemanla yer değiştirir.

Time Complexity: $O(n^2)$



Swaps

0

Comparisons

0



selection_sort.py

```
def selection_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        min_idx = i  
        for j in range(i+1, n):  
            if arr[j] < arr[min_idx]:  
                min_idx = j  
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```



▶ Play

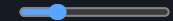


<https://gemini.google.com/share/484d47bb91e8>

Speed



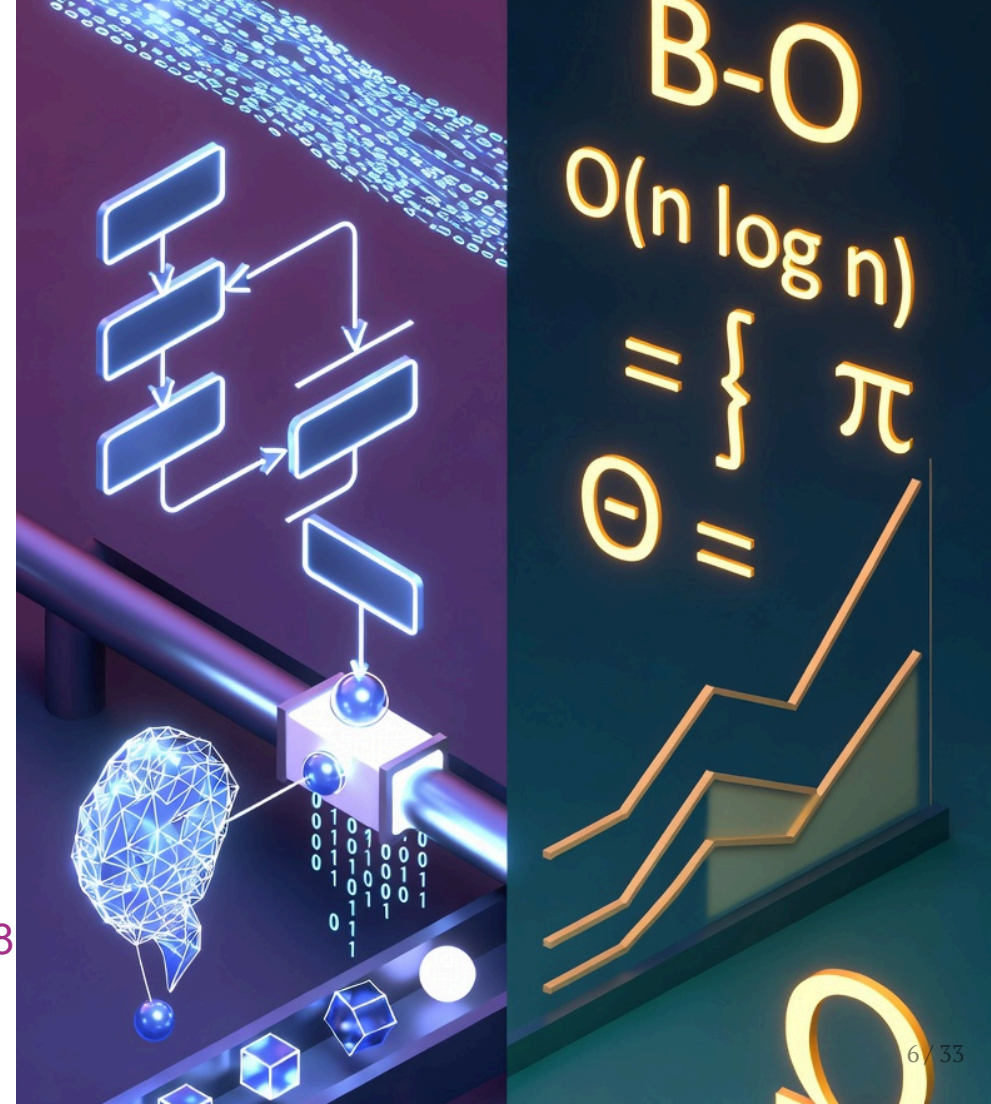
Elements



Selection Sort: Matematiksel Analiz

- Selection Sort, giriş verisinin baştaki diziliminden (sıralı/ters) tamamen bağımsızdır.
- Toplam karşılaştırma sayısı her koşulda deterministik olarak $n*(n-1)/2$ formülüne eşittir.
- Bu nedenle En İyi, En Kötü ve Ortalama durumların hepsi her zaman $\Theta(n^2)$ 'dir.
- Ancak toplam Swap (yer değiştirme) sayısı daima $O(n)$ ile sınırlıdır.

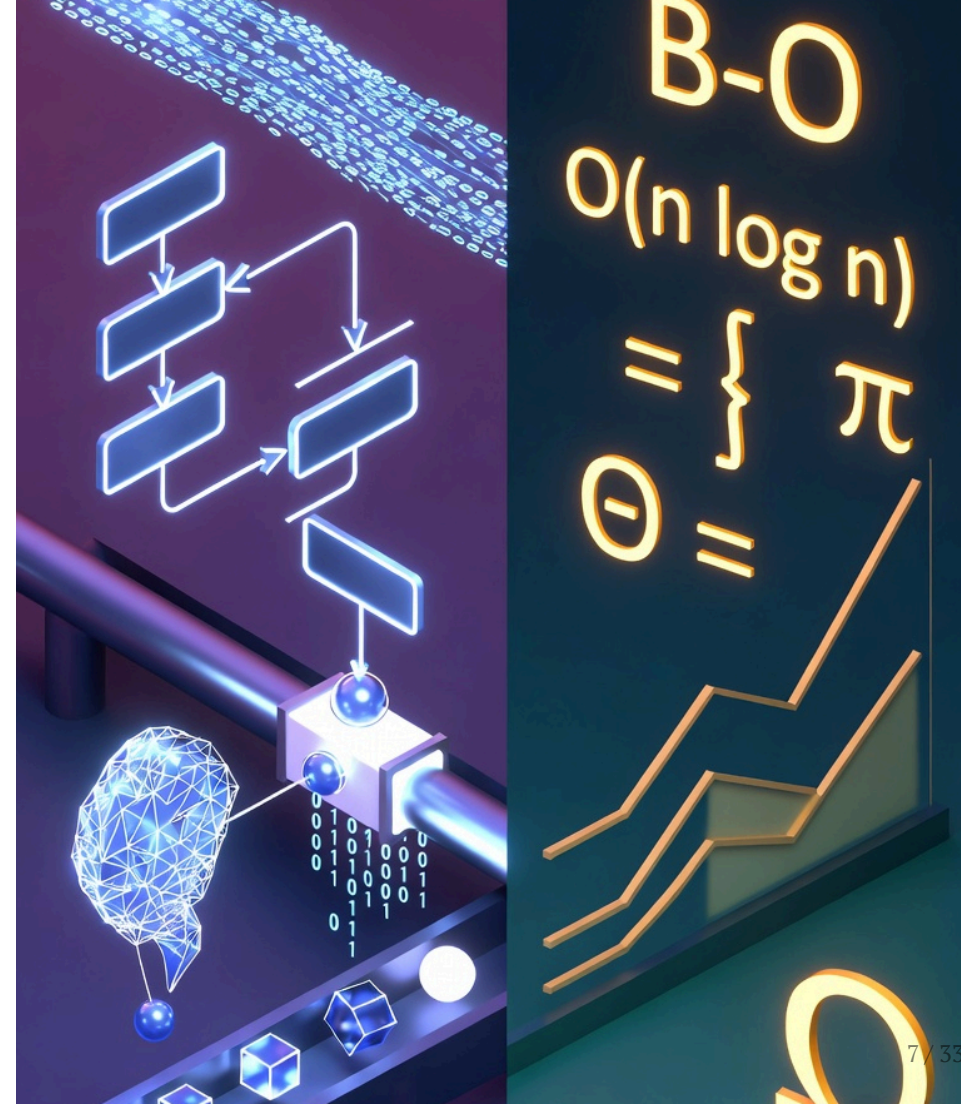
<https://gemini.google.com/share/484d47bb91e8>



Selection Sort: Pseudocode ve İzleme

- Sözde kod (Pseudocode), algoritmanın iç içe iki for döngüsünden oluştuğunu gösterir.
- Dış döngü, yerleştirilecek hedef pozisyonu (i) indeksini belirler.
- İç döngü (j), kalan sırasız bölümdeki en küçük elemanın (min_idx) konumunu arar.
- İç döngü bitince, i ile min_idx konumundaki veriler bellek üzerinde takas edilir.

<https://gemini.google.com/share/484d47bb91e8>



Algorithm

```
1  procedure selection_sort(A):
2      n = length(A)
3      for i = 0 to n - 2:
4          min_idx = i
5          for j = i + 1 to n - 1:
6              if A[j] < A[min_idx]:
7                  min_idx = j
8          swap(A[i], A[min_idx])
```

<https://gemini.google.com/share/484d47bb91e8>

Bubble Sort: Komşu Karşılaştırması

- Bubble Sort, ardışık (yan yana) duran komşu iki elemanı sürekli karşılaştırır.
- Eğer soldaki eleman sağdakinden büyükse (yanlış sıra), elemanlar yer değiştirir (swap).
- Bu süreç, en büyük elemanların su yüzeyine çıkan kabarcıklar gibi sona itilmesini sağlar.
- Büyük elemanların dizinin sağına doğru biriktirildiği artımlı bir yapı kurar.

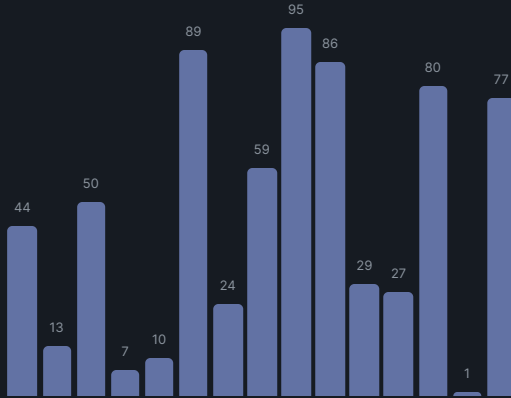
<https://gemini.google.com/share/484d47bb91e8>

Bubble Sort

Komşu elemanları karşılaştırarak büyük olanı sağa iter. Her turda en büyük eleman sona yerleşir.

Time Complexity:

$O(n^2)$



Swaps0

Comparisons0

bubble_sort.py

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
```



▶ Play

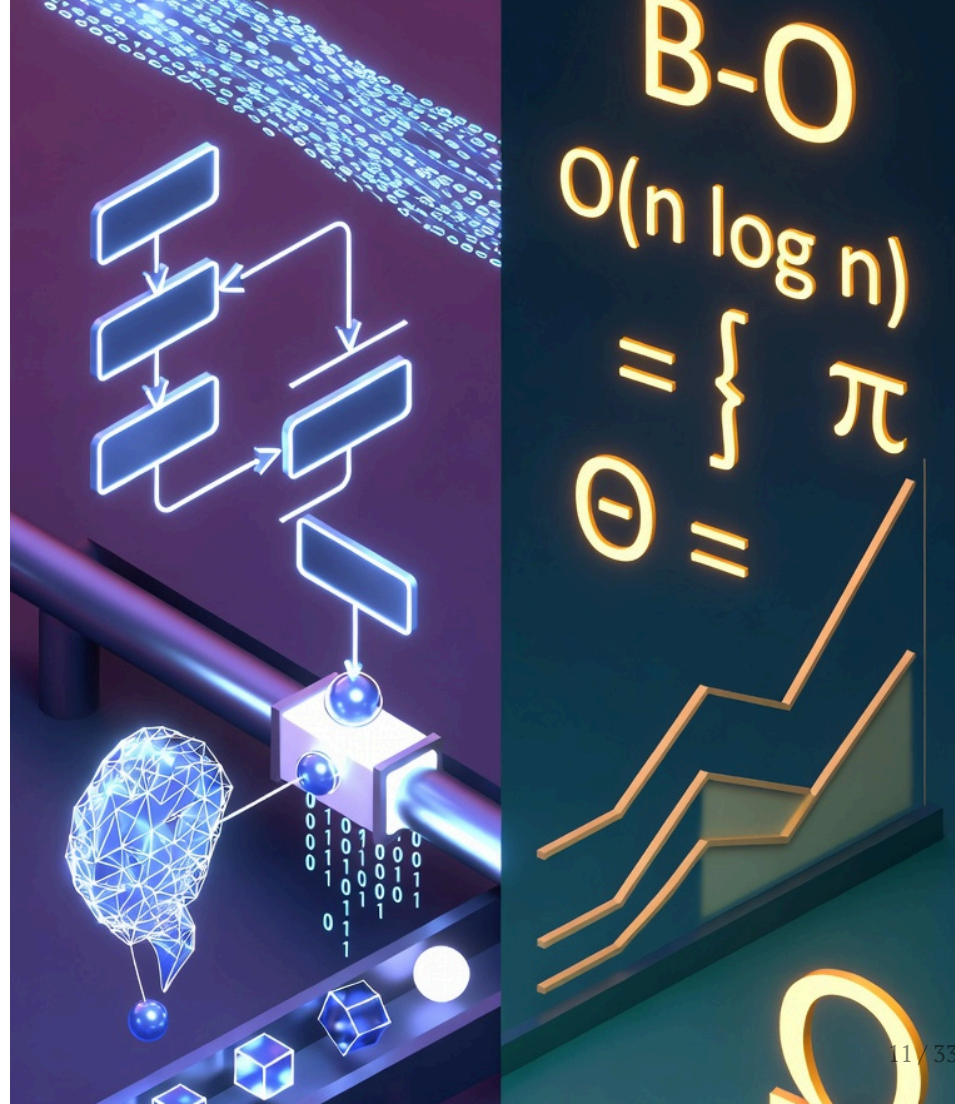


Speed

Elements

Bubble Sort: Optimizasyon (Flag Kullanımı)

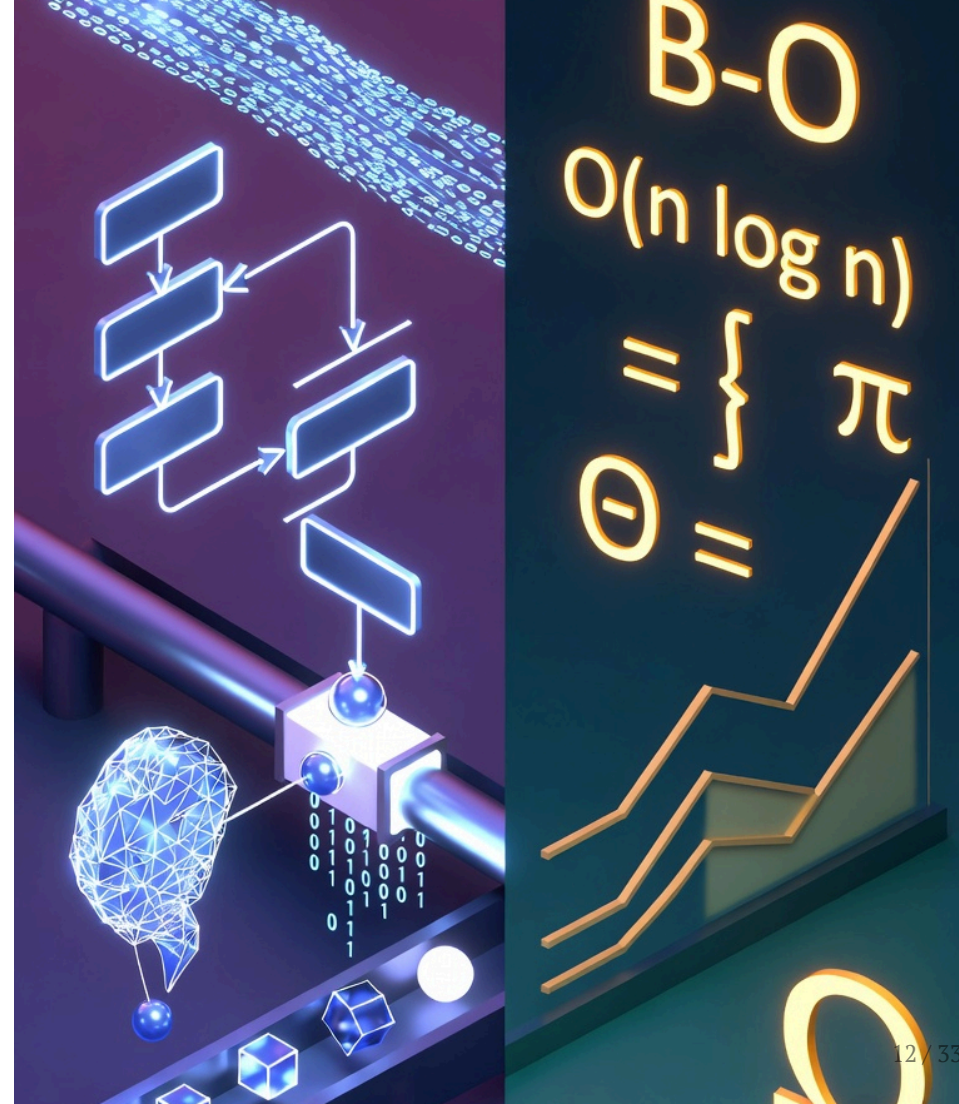
- Klasik Bubble Sort, dizi sıralansa bile n^2 adım boyunca çalışmaya devam eder.
- Akıllı optimizasyon: Bir 'swapped' (değişim yapıldı) bayrağı mantıksal kontrolcüsü eklenir.
- Eğer algoritma tam bir döngü turunda hiç yer değiştirme yapmadıysa dizi sıralıdır.
- Dizi zaten sıralıysa algoritma 'break' komutuyla erken sonlanarak zaman tasarrufu sağlar.



Bubble Sort: Zaman Karmaşıklığı Analizi

- Flag optimizasyonu yoksa, algoritmanın çalışma süresi her zaman kesin $\Theta(n^2)$ 'dir.
- Optimizasyon uygulandığında En İyi Durum (tamamen sıralı veri) $\Omega(n)$ olur.
- En Kötü Durum (ters sıralı veri), her adımda swap yapıldığı için $O(n^2)$ 'dir.
- Aşırı swap maliyeti nedeniyle, donanım üzerinde gerçek çalışma süresi Selection'dan uzundur.

<https://gemini.google.com/share/484d47bb91e8>



Insertion Sort: Aşamalı İnşa

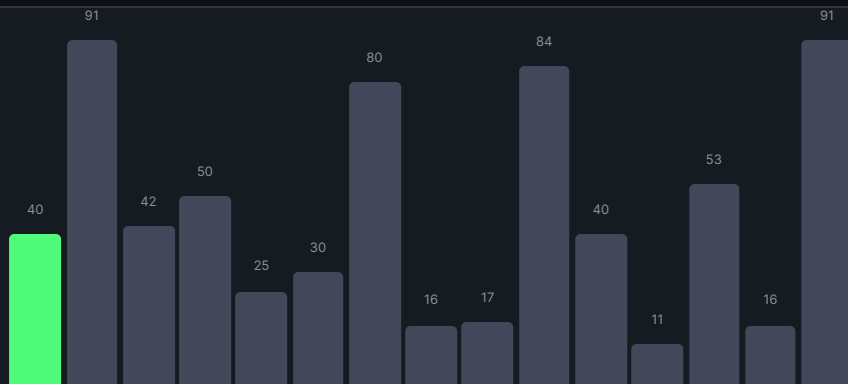
- Insertion Sort (Eklemeli Sıralama), 'Artımlı' (Incremental) tasarım yaklaşımının en doğal temsilcisidir.
- Dizi, dinamik olarak büyüyen sol sıralı bölge ve daralan sırasız bölgeden oluşur.
- Sırasız bölgeden çekilen her yeni eleman, sıralı bölgede doğru yere 'sokulur'.
- İskambil veya Rummy kartlarını elde dizerken insanların farkında olmadan kullandığı reflekstir.

<https://gemini.google.com/share/484d47bb91e8>

Insertion Sort

Time Complexity: $O(n^2)$

Her yeni elemanı, solundaki sıralı alt dizi içinde uygun pozisyona yerleştirerek ilerler.



Shifts0

Comparisons0

insertion_sort.py

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j >= 0 and key < arr[j]:  
            arr[j + 1] = arr[j]  
            j -= 1  
        arr[j + 1] = key
```



Play



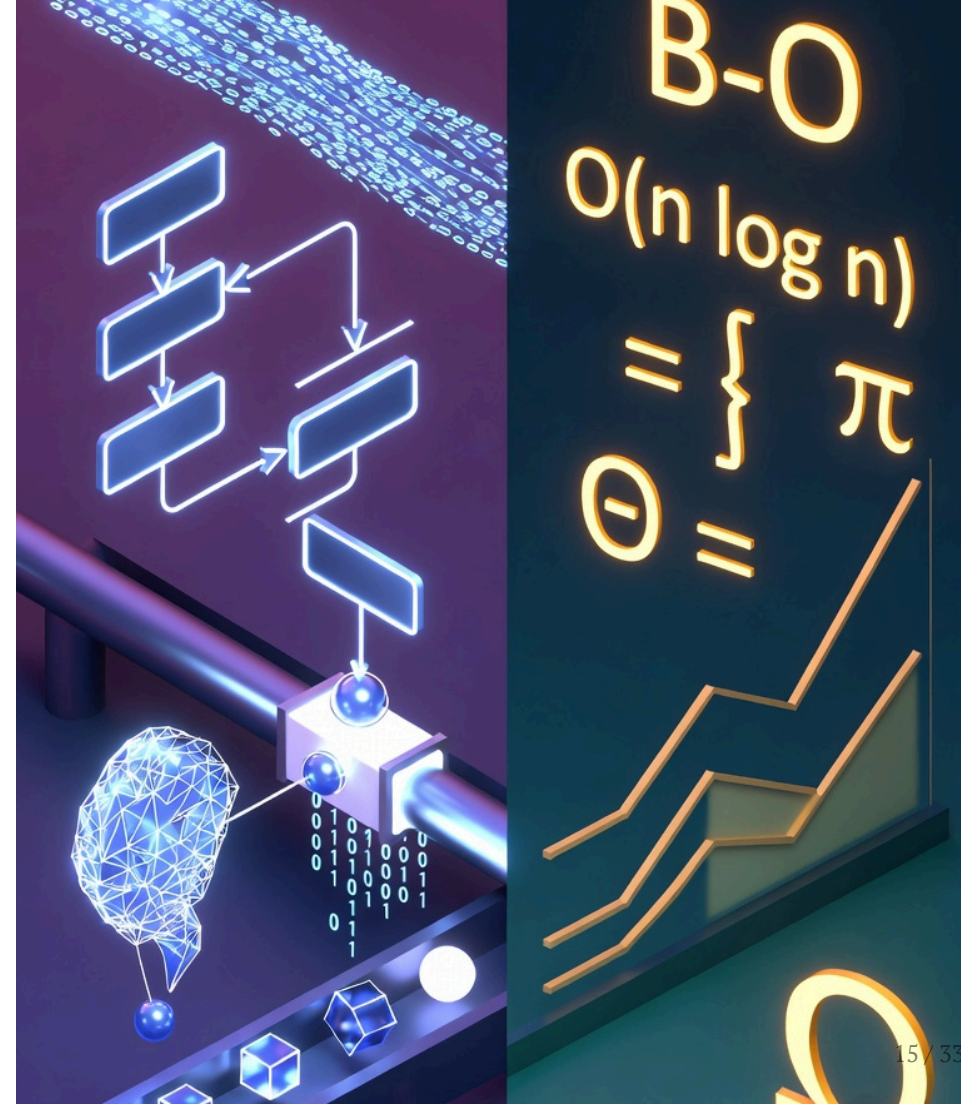
Speed

Elements

Insertion Sort: Teknik Detaylar

- Insertion Sort, elemanları yerleřtirmek için Swap (Takas) yerine Shift (Kaydırma) kullanır.
- Eklenmesi gereken yeni deęer bir 'key' (anahtar) deęiřkeninde geęici olarak saklanır.
- Sıralı bölgedeki büyük elemanlar tek tek saęa kaydırılarak yeni deęere boşluk açılır.
- Kaydırma iřlemi, üçlü deęiřken ataması gerektiren Swap iřleminden CPU seviyesinde daha ucuzdur.

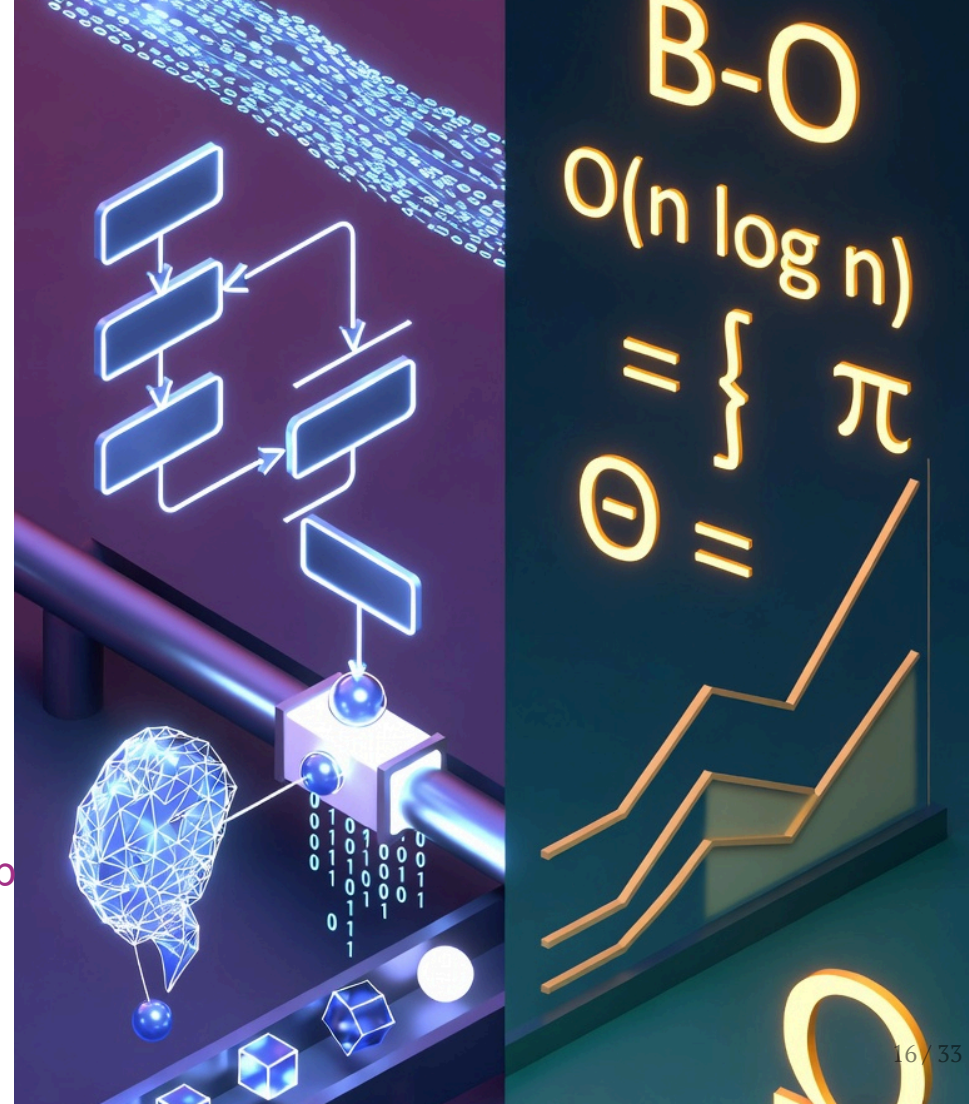
<https://gemini.google.com/share/2066e221005b>



Insertion Sort: En İyi Durum Analizi

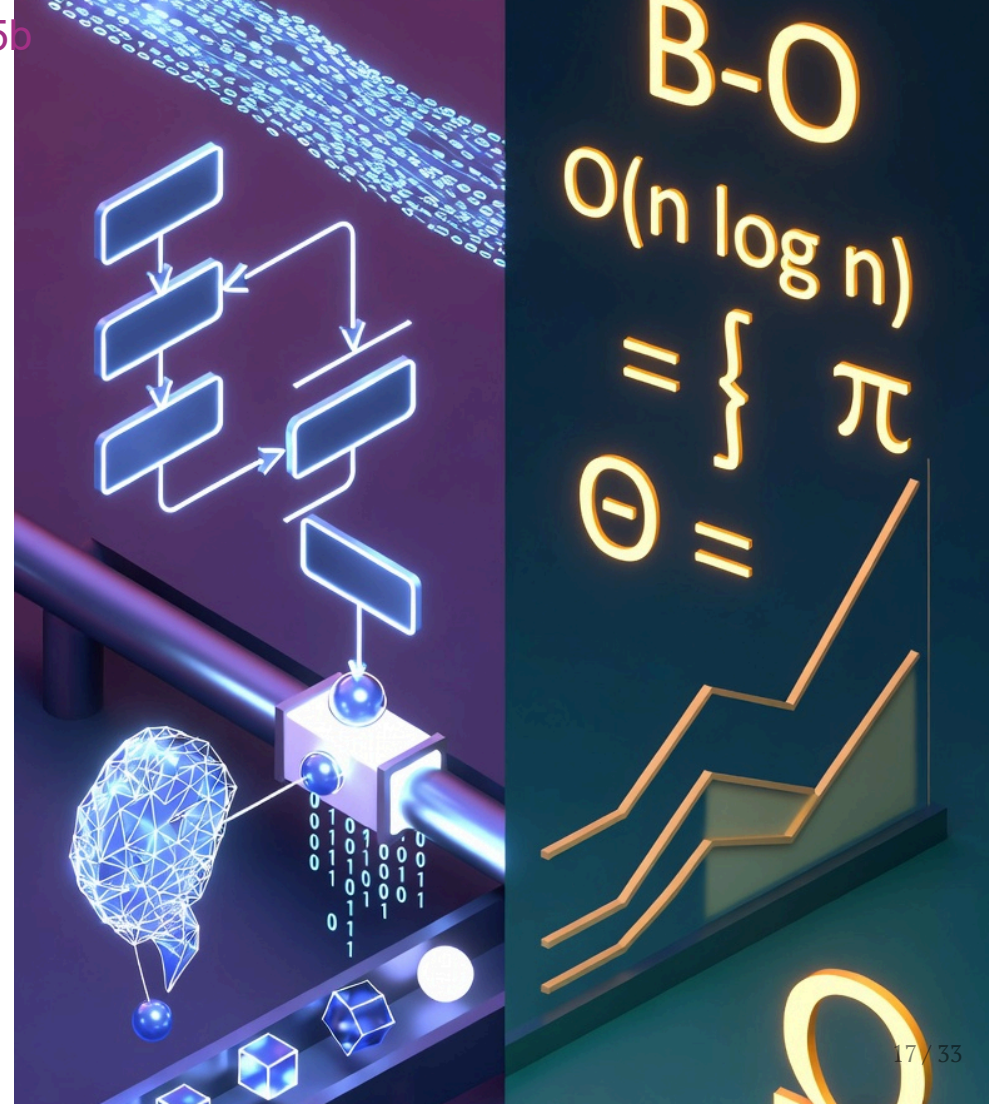
- Algoritmaya zaten tamamen sıralı bir dizi verildiğinde En İyi Durum (Best Case) oluşur.
- İç döngü (while), sadece bir kez karşılaştırma yapar ve hemen sonlanır.
- Bu senaryoda karmaşıklık, $n-1$ adım süren lineer zamanlı $O(n)$ performansına ulaşır.
- Sistemlere anlık akan canlı verilerin (stream) sıralanmasında muazzam bir avantaj sunar.

<https://gemini.google.com/share/2066e221005b>



Insertion Sort: En Kötü Durum Analizi

- Algoritmaya tamamen ters (büyükten küçüğe) sıralı dizi verildiğinde En Kötü Durum oluşur.
- Her yeni eleman için sıralı bölgedeki tüm elemanların sağa kaydırılması zorunlu hale gelir.
- Kaydırma ve karşılaştırma sayısı Gauss toplamına $(n*(n-1)/2)$ denk düşerek $O(n^2)$ olur.
- Bu durum, karesel algoritmaların rastgele verilerdeki kronik darboğazını temsil eder.



Selection vs Insertion Sort Karşılaştırması

- Selection Sort diziyi 'körlemesine' tarar; verinin yapısından asla performans çıkarımı yapmaz.
- Insertion Sort 'adaptif'tir; sıralılık oranını fark edip döngüleri erken keserek hızlanır.
- Belleğe veri yazma (Swap) maliyeti çok yüksekse Selection Sort $O(n)$ ile daha iyidir.
- Veri akışı anlık (online) geliyorsa ve neredeyse sıralıysa tartışmasız Insertion seçilir.

<https://gemini.google.com/share/2066e221005b>

Sıralama Algoritmalarında Stable Kavramı

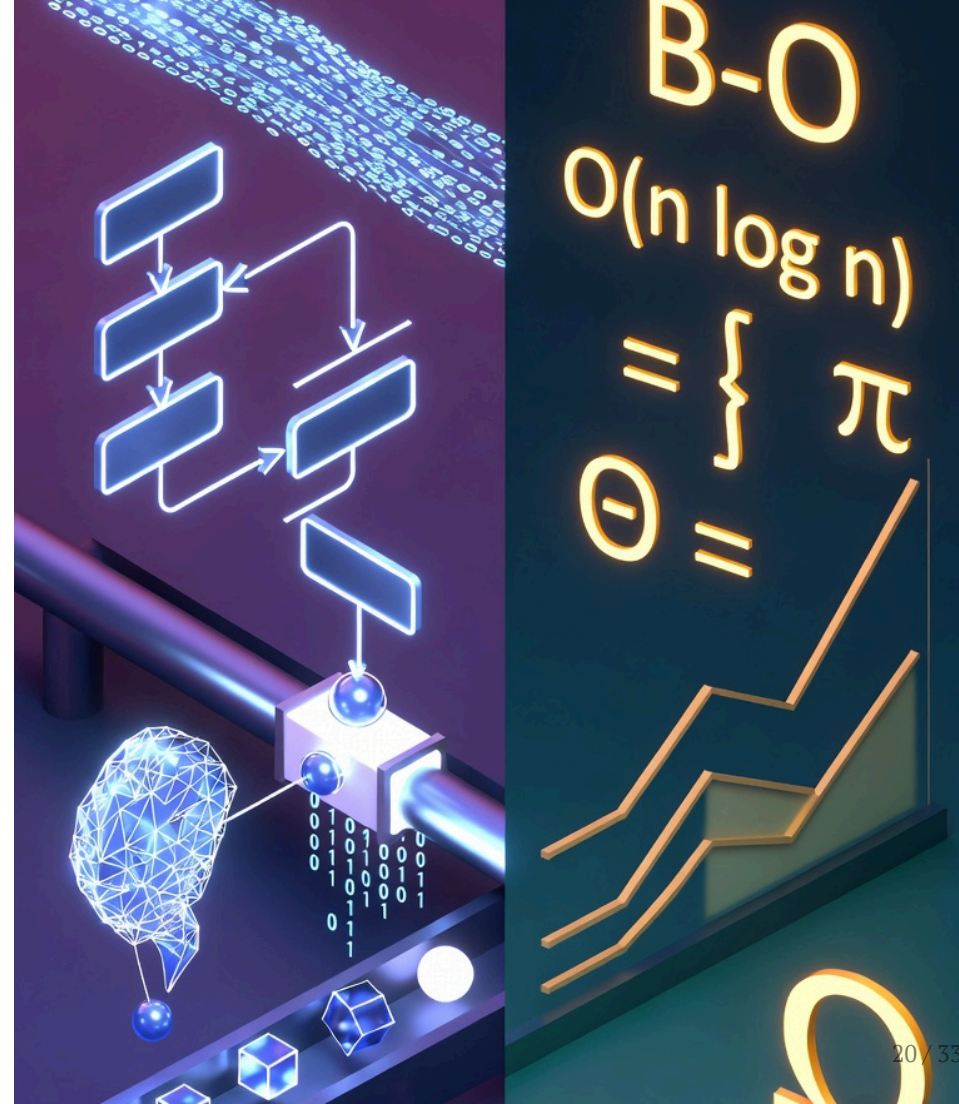
Derinlemesine

- Insertion ve Bubble Sort ardışık elemanları kaydırıldığı için doğası gereği kararlıdır (Stable).
- Selection Sort ise uzak indeksler arası Swap yaptığı için kararlılığı (Stability) bozar.
- Örnek: 4a, 4b, 2 kümesinde Selection, 4a ile 2'yi değiştirip sırayı 2, 4b, 4a yapar.
- Object (Nesne) yönelimli programlamada nesne referans sıralamasında bu fark hayatidir.

<https://gemini.google.com/share/2066e221005b>

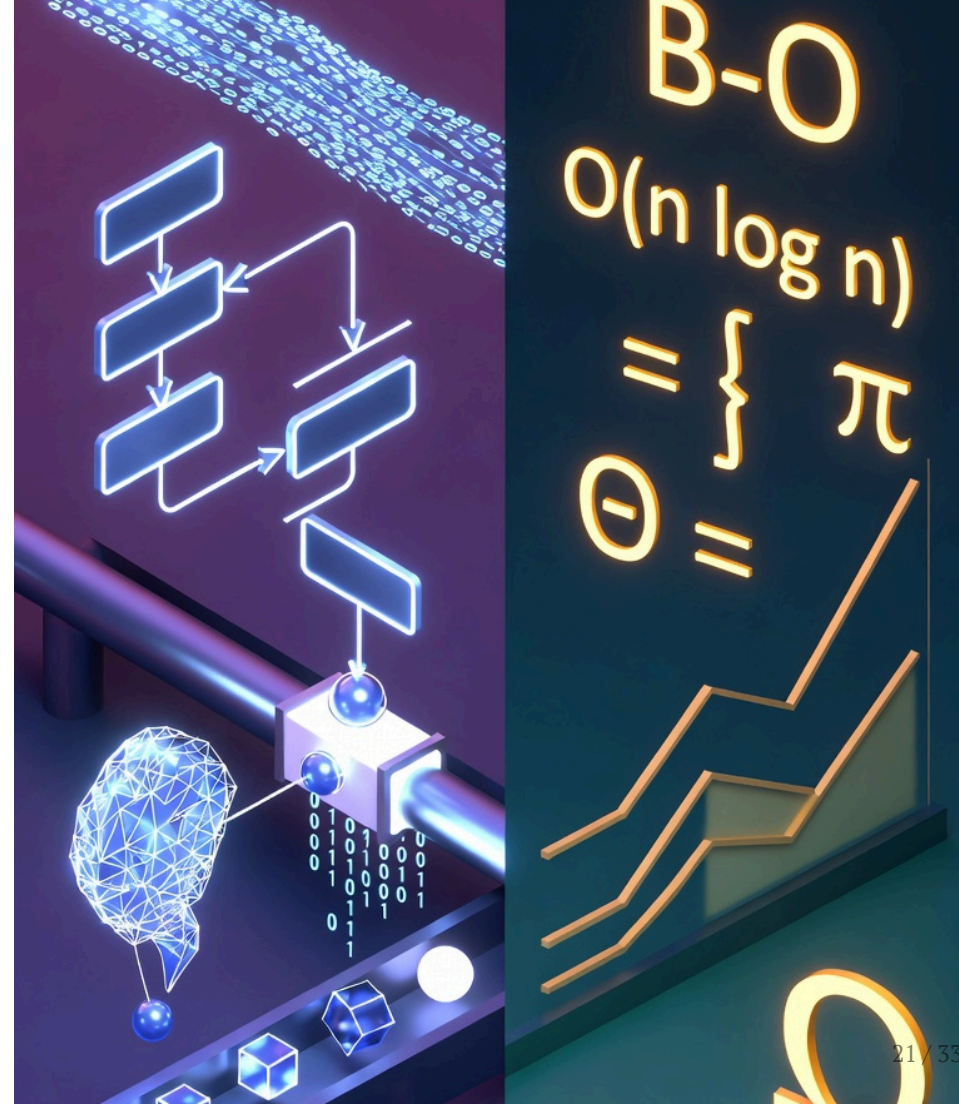
Döngü Değişmezi (Loop Invariant) Kavramı

- Döngü Değişmezi, karmaşık algoritmaların her zaman doğru çalıştığını ispatlayan matematiksel araçtır.
- Başlangıç (Initialization): Döngü hiç başlamadan önce belirlenen kuralın doğru olması durumu.
- Koruma (Maintenance): Döngünün bir turu bittiğinde kuralın bir sonraki tur için hala geçerli olması.
- Sonlanma (Termination): Döngü tamamen bittiğinde, problemin çözüldüğünün



Insertion Sort için Döngü Değişmezi Analizi

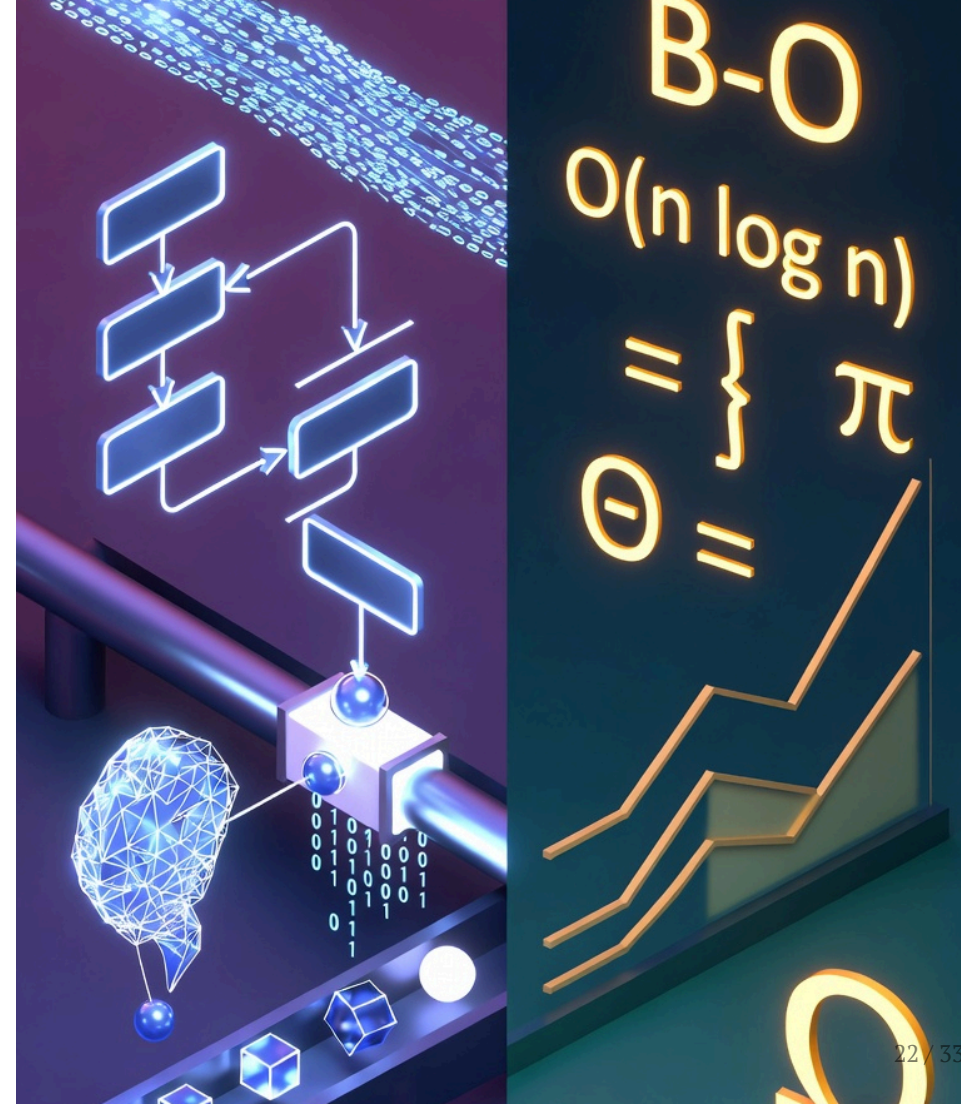
- İddia: Dış for döngüsünün her başında, $A_0 \dots i-1$ alt dizisi kesinlikle sıralı bir bloktur.
- Başlangıç ($i=1$): $A_0 \dots 0$ tek bir elemandır ve matematiğe göre tek eleman her zaman sıralıdır.
- Koruma: İç döngü, A_i elemanını sol taraftaki bu sıralı yapının içine bozmadan entegre eder.
- Sonlanma: $i=n$ olduğunda, $A_0 \dots n-1$ yani tüm dizi hatasız bir şekilde sıralanmış olur.



Küçük Veri Setlerinde Performans ($n < 15$)

- Asimptotik analiz $O(n \log n)$ harika dese de, pratikte gizli sabitler (overhead) vardır.
- n değerinin 10-15'ten küçük olduğu durumlarda karesel olan Insertion Sort çok daha hızlıdır.
- Recursive (özyinelemeli) algoritmaların fonksiyon çağırma (Call Stack) maliyetleri çok ağırdır.
- Java, Python (Timsort) kütüphaneleri, dizileri bölerken küçük parçalarda Insertion'a geçer.

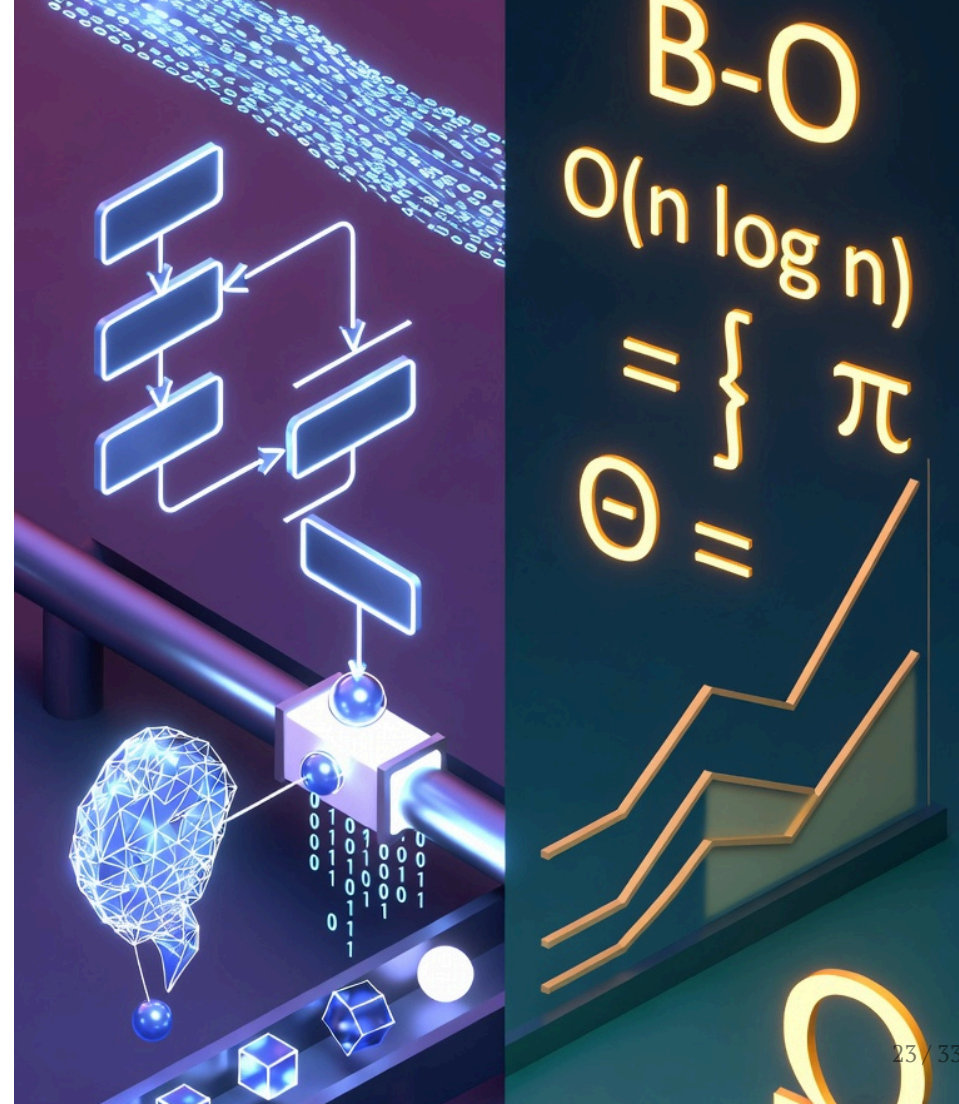
<https://gemini.google.com/share/2066e221005b>



Bellek Hiyerarşisi ve Sıralama

- İşlemcinin veriye ulaşma hızı, algoritmanın Big-O karmaşıklığı kadar belirleyicidir.
- Önbellek (Cache) dostu algoritmalar, belleğe fiziksel olarak yan yana (ardışık) erişenlerdir.
- Insertion Sort ardışık Shift yaptığı için L1/L2 Cache locality (yerellik) prensibine tam uyar.
- Selection Sort uzak noktalara atladığı için Cache Miss (önbellek ıskası) yaşatarak sistemi yavaşlatır.

<https://gemini.google.com/share/2066e221005b>



Selection Sort Pseudocode İncelemesi

- Pseudocode'da dizinin son elemanına $(n-1)$ dış döngünün gitmemesinin nedeni optimizasyondur.
- Geriye tek bir sırasız eleman kaldığında, o eleman mecburen en büyüktür ve yeri doğrudur.
- `min_idx` değişkeni değeri değil, minimum elemanın bellek adresini (indeksini) işaret eder.
- Kodun deterministik doğası, çalışma süresini tahmin edilebilir kılan gömülü sistemler için idealdir.

Algorithm

```
1  procedure selection_sort(A):
2      n = length(A)
3      for i = 0 to n - 2:
4          min_idx = i
5          for j = i + 1 to n - 1:
6              if A[j] < A[min_idx]:
7                  min_idx = j
8          swap(A[i], A[min_idx])
```

<https://gemini.google.com/share/2066e221005b>

Bubble Sort Pseudocode İncelemesi

- Dış döngünün görevi sadece gerekli iterasyon sayısını (tur sayısını) yönetmektir.
- İç döngü $(n-i-2)$ 'ye kadar gider; çünkü her turda en büyükler sağa birikir.
- 'swapped' boolean bayrağı, sıralılık durumunu anlık olarak (real-time) dinleyen dedektördür.
- If bloğundaki koşul $(A_j > A_{j+1})$, algoritmanın Stability (kararlılık) kuralının kilididir.

Algorithm

```
1  procedure bubble_sort(A):
2      n = length(A)
3      for i = 0 to n - 2:
4          swapped = False
5          for j = 0 to n - i - 2:
6              if A[j] > A[j+1]:
7                  swap(A[j], A[j+1])
8                  swapped = True
9          if not swapped:
10             break
```

<https://gemini.google.com/share/2066e221005b>

Insertion Sort Pseudocode İncelemesi

- Dış döngü $i=1$ 'den başlar; çünkü $i=0$ indeksindeki ilk eleman zaten yapısal olarak sıralıdır.
- Geçici 'key' değişkeni veriyi yedeklerken, 'j' değişkeni sıralı bölgede geri sayım yapar.
- While şartındaki ($A_j > \text{key}$), büyük elemanları sağa kaydıran (shift) ana mekanizmadır.
- While döngüsü bittiğinde j'nin bir eksikğine değil, $j+1$ 'e yazılması algoritmanın püf noktasıdır.

Algorithm

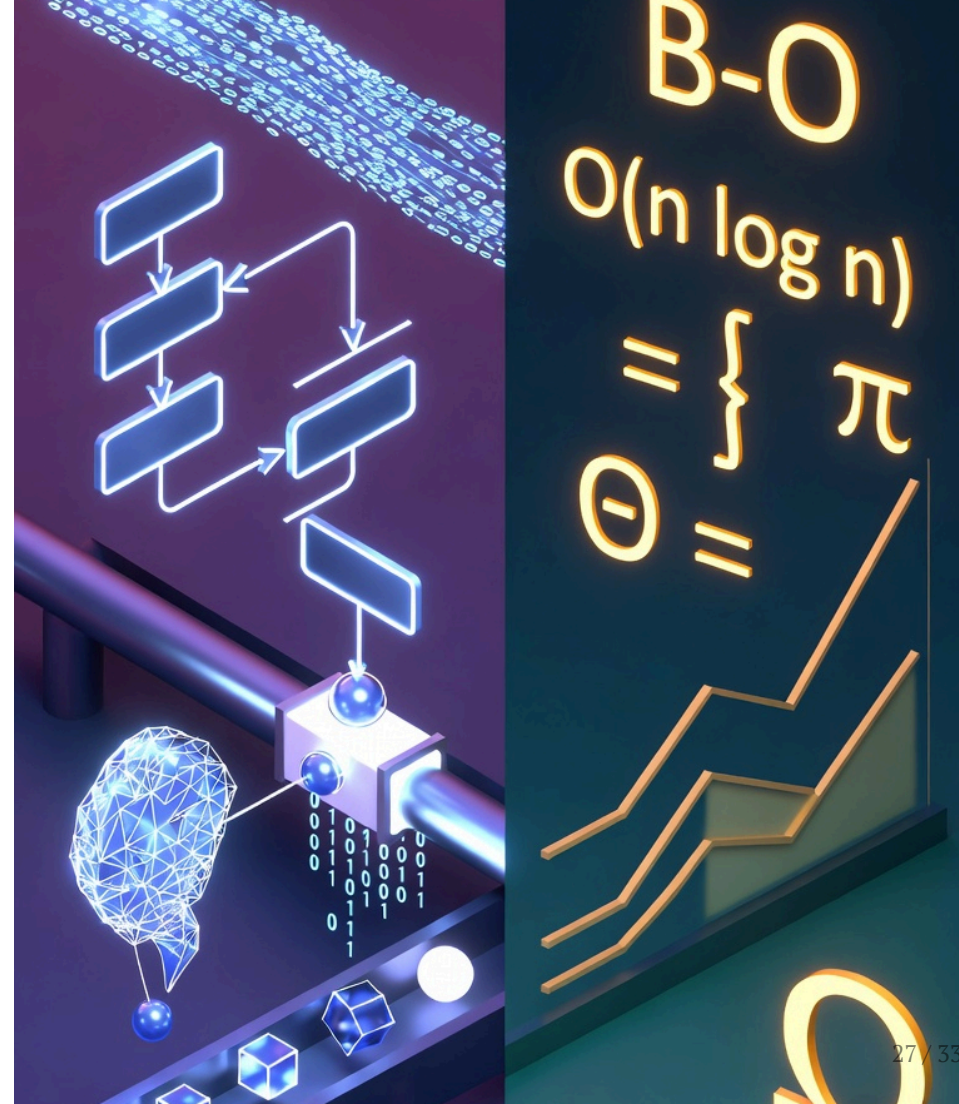
```
1  procedure insertion_sort(A):
2      n = length(A)
3      for i = 1 to n - 1:
4          key = A[i]
5          j = i - 1
6          while j >= 0 and A[j] > key:
7              A[j+1] = A[j]
8              j = j - 1
9          A[j+1] = key
```

<https://gemini.google.com/share/2066e221005b>

Karesel Karmaşıklık (n^2) Neden Kötüdür?

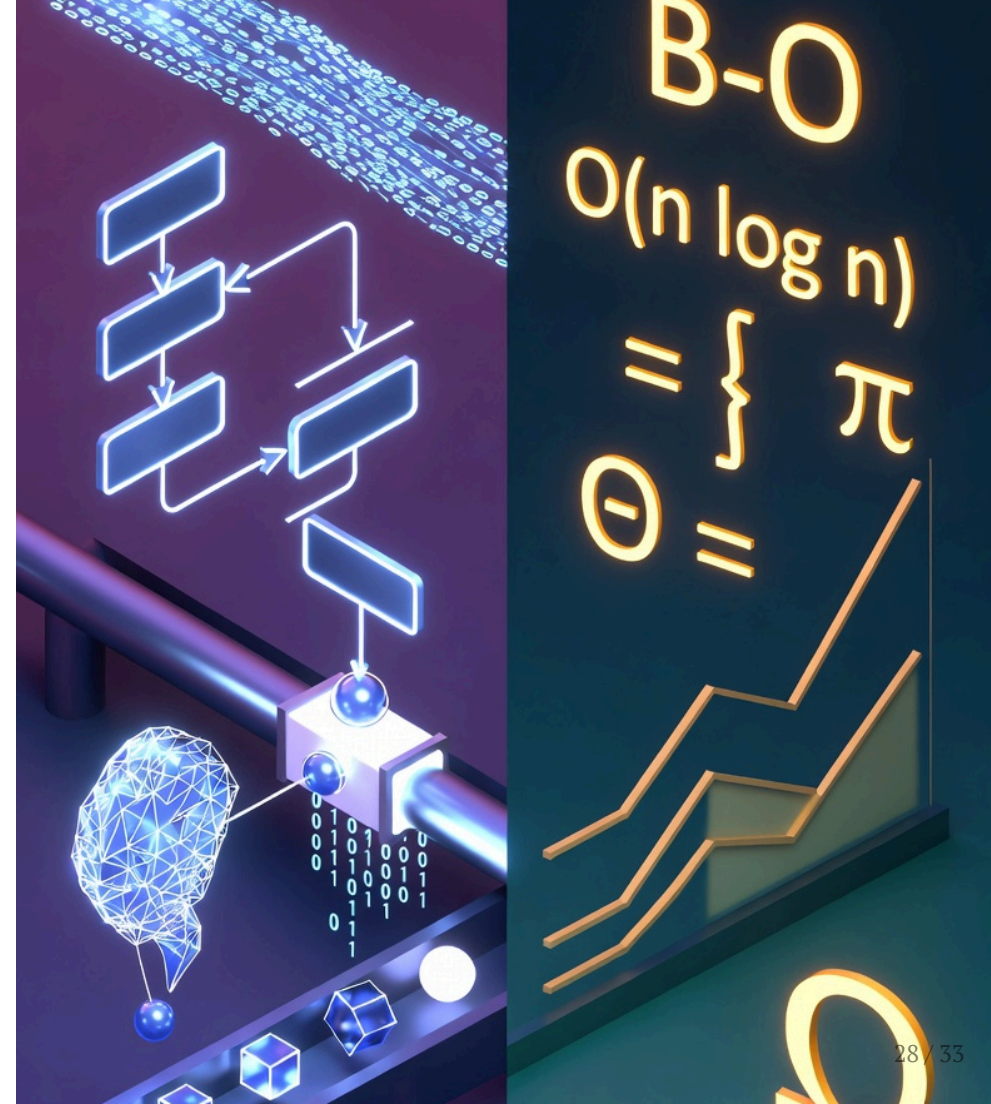
- Karesel fonksiyonda, veri boyutu 10 katına çıktığında işlem süresi 100 kat artar.
- $n=1.000.000$ veri setinde (Big Data) karesel algoritma 1 trilyon işlem adımı gerektirir.
- Donanım (CPU/RAM) hızları artsa da, üstel büyüme karakteristiği donanımı her zaman çaresiz bırakır.
- Modern yazılım mimarilerinde amaç, problemleri parçalayarak $O(n \log n)$ sınırına çekmektir.

<https://gemini.google.com/share/2066e221005b>



Hangi Algoritma Ne Zaman Seçilmeli?

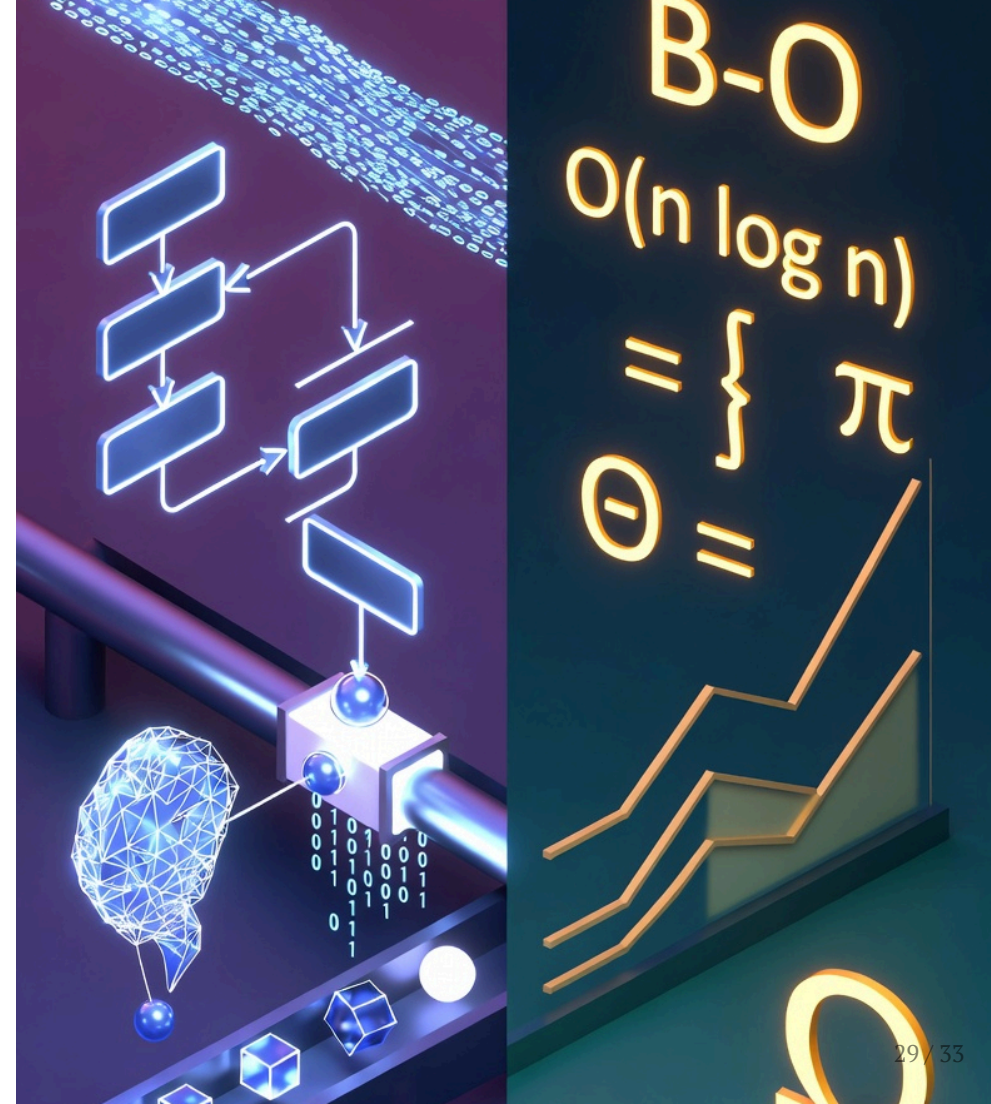
- Neredeyse sıralı (nearly sorted) veya anlık akan (streaming) veriler için daima Insertion Sort.
- Belleğe veri yazma (flash bellek yıpranması vb.) maliyeti çok kritiksə Selection Sort.
- Sıralama yanında kararlılık (stability) istenmiyorsa Selection, mutlaka isteniyorsa Insertion.
- Sadece eğitim ve algoritma mantığı (swap/flag) gösterimi amaçlanıyorsa Bubble Sort.



Sıralama Algoritmaları Görselleştirme Özeti

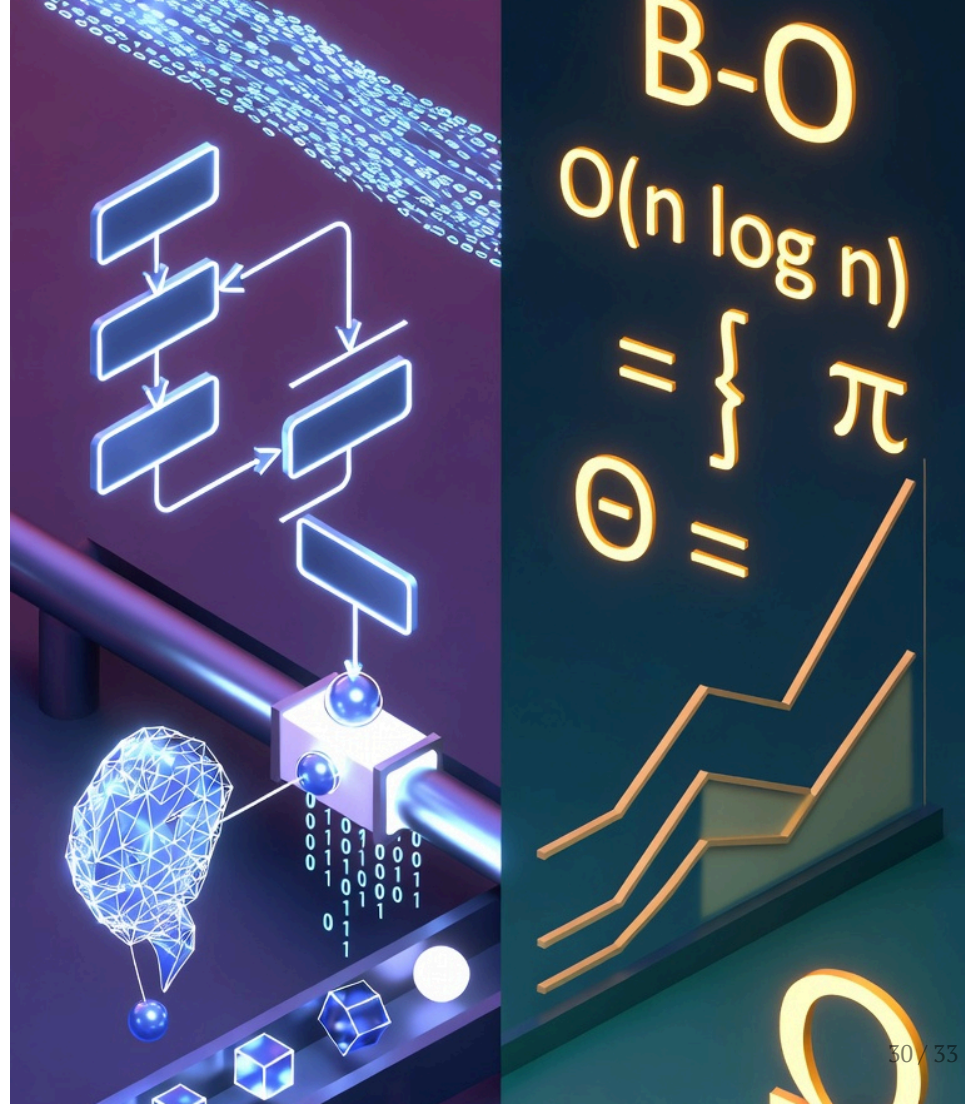
- Selection Sort: Dizinin en solundan başlayarak sağa doğru büyüyen kesin sıralı bir blok çizer.
- Bubble Sort: Dizinin en sağından başlayarak sola doğru büyüyen kesin sıralı bir blok çizer.
- Insertion Sort: Solda sıralı blok çizer ama içine yeni eleman girdikçe dinamik değişir.
- Görsel gelişim yönleri (Soldan/Sağdan), algoritmaların ezberlenmesinde çok güçlü bir akılda tutucu araçtır.

<https://gemini.google.com/share/2066e221005b>



Hafta 2 Özeti ve Gelecek Haftaya Bakış

- Karesel algoritmalar $O(n^2)$, veri hareketini ve In-place/Stable kavramlarını anlamak için temel oluşturdu.
- Selection, Bubble ve Insertion Sort'un donanım ve veri düzeneği karşısındaki zaaflarını öğrendik.
- Devasa veri setleri için, problemi parçalayarak bölen yeni zihniyetlere (Divide and Conquer) ihtiyaç var.
- Gelecek Hafta: Karesel duvarı aşan logaritmik hızlı (Merge/Quick) sıralama mimarilerini inşa edeceğiz.



Soru 1

<https://gemini.google.com/share/2066e221005b>

Selection Sort algoritması neden "kararsız" (unstable) olarak kabul edilir? Bir örnekle açıklayınız.

Çözüm Adımları

- **Kararlılık Tanımı:** Sıralama sonrasında, aynı değere sahip elemanların birbirlerine göre olan orijinal sıralarının korunması durumudur.
- **Neden Kararsız?** Selection Sort, her adımda en küçük elemanı bulur ve mevcut pozisyondaki elemanla **uzak mesafeli bir takas (swap)** yapar. Bu takas, aradaki aynı değerli elemanların sırasını bozabilir.
- **Örnek Senaryo:** $[2a, 2b, 1]$ dizisini düşünelim.
 - Algoritma dizideki en küçük elemanı (1) bulur.
 - 1 ile en baştaki $2a$ yer değiştirir.
- **Sonuç:** Yeni dizi $[1, 2b, 2a]$ olur. Dikkat edilirse $2a$, kendisiyle aynı değerdeki $2b$ ve $2a$ 'nin **arkasına** düşmüştür.
- **Karar:** Başlangıçtaki $a \rightarrow b$ sırası bozulduğu için algoritma **unstable** (kararsız) olarak sınıflandırılır.

Learning

Learning Outcome: Algoritma kararlılığı (stability) analizi

Soru 2

Insertion Sort algoritmasının "en iyi durum" (best case) zaman karmaşıklığı nedir ve bu hangi tür veri setlerinde gerçekleşir?

Çözüm Adımları

- **En İyi Senaryo:** Girdi dizisinin zaten **tamamen sıralı** (veya sıralıya çok yakın) olması durumudur.
- **Döngü Mekanizması:** Dış döngü her zaman n defa döner. Ancak içteki `while` döngüsü, elemanların yerini kontrol eder.
- **Erken Çıkış:** Dizi sıralı olduğunda, `while` döngüsünün koşulu ($A[j] > key$) ilk kontrolde bozulur ve iç döngü gövdesi hiç çalışmaz.
- **Hesaplama:** Sadece dış döngünün maliyeti kalır: $1 + 1 + \dots + 1$ (n kez).
- **Sonuç:** Toplam işlem sayısı n ile doğru orantılıdır, bu nedenle karmaşıklık $O(n)$ (Lineer Zaman) olur.

Learning Outcome: Adaptif algoritma analizi

Soru 3

<https://gemini.google.com/share/2066e221005b>

Dizinin "**neredeyse sıralı**" olduğu (her elemanın hedef yerinden en fazla k birim uzakta olduğu) bir durumda en verimli temel sıralama algoritması hangisidir?

Çözüm Adımları

- **Selection Sort:** Verinin durumuna bakmaksızın her zaman en küçüğü aradığı için performansı değişmez ve $O(n^2)$ çalışır.
- **Bubble Sort:** Erken çıkış (flag) özelliği olsa bile, elemanları uzak mesafelere taşımak için çok sayıda gereksiz takas yapar.
- **Insertion Sort Mekanizması:** Her eleman hedef konumundan en fazla k birim uzaktaysa, iç döngü her eleman için en fazla k defa çalışır.
- **Karmaşıklık Analizi:** Toplam maliyet $O(n \cdot k)$ olur. Eğer k değeri n 'den çok küçük bir sabitse ($k \ll n$), bu performans pratik olarak $O(n)$ seviyesine yaklaşır.
- **Cevap:** Bu özel veri karakteristiği için en verimli temel algoritma **Insertion Sort**'tur.

Learning Outcome: Veri karakteristiğine göre algoritma seçimi